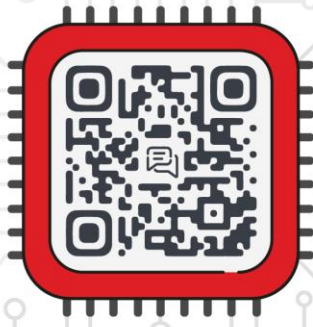




نظري

11

13

1650 sp

كلية الهندسة المعلوماتية

السنة الرابعة

البنيان المؤسساتي والبرمجة
بالمكونات وتطوير الخدمات
في الانظمة الموزعة

د. عمار جوخدار

محتوى مجاني غير مخصص للبيع التجاري



RB Informatics 22/6/2026

برمجة تفرعية

مقدمة :

- توجد عدة أنواع من البنى البرمجية (Software Architectures)، ويختلف اختيار البنيان المناسب باختلاف نوع التطبيق ومتطلباته. فالتطبيقات الرسومية (Graphics Applications)، وتطبيقات الواقع الافتراضي (Virtual Reality - VR)، والألعاب، وتطبيقات الويب، جميعها تمتلك أنماطاً معمارية مختلفة صُممت لتلائم طبيعة عملها والخصائص التي تحتاج إلى تحقيقها.
- ويُعد البنيان المؤسساتي حجر الأساس في تطبيقات الويب (Client-side & Server-side)، لأنه لا يهتم بالوظائف التي يقدمها التطبيق للمستخدم فحسب، وإنما **يوفر البنية اللازمة لتحقيق الخصائص غير الوظيفية** التي يصعب إضافتها لاحقاً، مثل إدارة الموارد، والمناقلات (Transactions)، والأمان، والاعتمادية، وقابلية التوسع، وتكامل الأنظمة.
- إن عدم الالتزام ببنيان موحد داخل المؤسسة يؤدي إلى ظهور مشكلات كبيرة عند محاولة دمج التطبيقات المختلفة. فقد يعمل كل برنامج بصورة صحيحة عند تشغيله بشكل مستقل، لكنه يفشل عند تكامله مع بقية الأنظمة بسبب اختلاف أساليب إدارة الموارد والبنية الداخلية لكل تطبيق.
- ومن الأمثلة على ذلك إدارة الخيوط (Threads)، حيث قد يعتمد كل تطبيق على آلية مختلفة لإنشاء الخيوط أو إدارة تجمعات الاتصالات (Connection Pools). وعند دمج هذه التطبيقات، قد يقوم كل خيط بإنشاء عدد إضافي من الخيوط في الخدمات الأخرى، مما يؤدي إلى تضخم عدد الخيوط العاملة داخل النظام، وزيادة استهلاك الموارد، وانخفاض الأداء، بل وقد يصل الأمر إلى حدوث تعارضات تمنع التطبيقات من العمل معاً بصورة صحيحة.
- وتظهر المشكلة نفسها في إدارة الاتصالات مع قواعد البيانات. فإذا استخدم أحد التطبيقات تجمع اتصالات (Connection Pool) مختلفاً عن بقية النظام، أو أنشأ اتصالات مباشرة خارج البنية المعتمدة، فقد تتعطل آليات إدارة المناقلات (Transactions)، لأن العمليات لن تعمل ضمن السياق (Transaction Context) نفسه، مما يؤدي إلى فقدان الاتساق بين مكونات النظام.
- لذلك لا يكفي أن يكون كل برنامج صحيحاً من الناحية البرمجية، بل يجب أيضاً أن يلتزم جميع أفراد الفريق بالبنيان المؤسساتي نفسه وبالآليات الموحدة لإدارة الخيوط، والاتصالات، والمناقلات، والموارد المشتركة، حتى تتمكن التطبيقات من العمل معاً بكفاءة وموثوقية.

■ ومع انتشار أدوات الذكاء الاصطناعي المساعدة في البرمجة، أصبح الالتزام بالبنية المعمارية أكثر أهمية من أي وقت مضى. فقد تستطيع هذه الأدوات إنتاج شيفرة برمجية صحيحة لتنفيذ وظيفة معينة، لكنها قد تستخدم أساليب مختلفة لإدارة الخيوط أو الاتصالات أو الوصول إلى قواعد البيانات إذا لم توجّه وفق البنيان المعتمد في المشروع.

■ لذلك يبقى فهم المهندس للبنية المعمارية للنظام ضرورياً لضمان تكامل جميع المكونات والمحافظة على الخصائص غير الوظيفية التي يعتمد عليها النظام بأكمله.

■ ثانياً: تحديات البرمجة بأدوات الذكاء الاصطناعي

■ شهدت السنوات الأخيرة انتشاراً واسعاً لأدوات الذكاء الاصطناعي المساعدة في البرمجة، مثل Claude و GitHub Copilot، وأصبحت قادرة على توليد كميات كبيرة من الشيفرة البرمجية خلال وقت قصير. ورغم الفوائد الكبيرة التي تقدمها هذه الأدوات، فإن الاعتماد عليها بصورة كاملة دون فهم للبنية المعمارية (Software Architecture) قد يؤدي إلى مشكلات يصعب اكتشافها ومعالجتها لاحقاً.

■ تكمن المشكلة الأساسية في أن أدوات الذكاء الاصطناعي تستطيع إنتاج أجزاء برمجية تعمل بصورة صحيحة عند اختبارها بشكل منفصل، لكنها لا تمتلك فهماً كاملاً للبنية الكلية للنظام. فهي تعالج كل جزء باعتباره وحدة مستقلة، بينما يعتمد نجاح الأنظمة الكبيرة على كيفية ترابط جميع هذه الأجزاء ضمن معمارية موحدة تحقق الخصائص غير الوظيفية للنظام.

■ يمكن تشبيه ذلك ببناء منزل دون وجود مهندس معماري يشرف على التصميم الكامل؛ فإذا تولى كل عامل بناء غرفة مستقلة اعتماداً على تعليمات منفصلة، فقد تكون كل غرفة سليمة بحد ذاتها، لكن عند تجميعها تظهر مشكلات إنشائية عديدة، مثل تكرار الجدران أو سوء توزيع الأحمال أو زيادة الوزن الكلي للبناء. وبالمثل، قد ينتج عن توليد كل جزء من التطبيق بصورة منفصلة تكرار في المكونات البرمجية، وزيادة غير مبررة في حجم النظام، وانخفاض في كفاءته وأدائه.

■ **ومن الملاحظات المهمة:** أن هذه الأدوات تهتم غالباً بالجوانب المحلية للتطبيق، فتنتج واجهات استخدام احترافية وشيفرة تبدو منظمة وحديثة، إلا أنها لا تضمن توافق هذه الأجزاء مع بقية مكونات النظام أو مع المعمارية المعتمدة، وهو ما يؤدي إلى تراكم التعقيد البرمجي مع مرور الوقت.

■ لهذا السبب بدأت كثير من الشركات تلاحظ أن الاعتماد المكثف على أدوات الذكاء الاصطناعي لم يحقق دائماً النتائج المتوقعة من حيث زيادة سرعة التطوير أو تحسين جودة البرمجيات. ففي كثير من الحالات أصبح حجم الشيفرة البرمجية أكبر وأكثر تعقيداً، وأصبح من الصعب على المطورين فهمها أو صيانتها، على الرغم من أن أجزاءها تعمل بصورة صحيحة عند اختبارها بشكل منفصل.

■ كما أن قدرة هذه الأدوات تعتمد بدرجة كبيرة على الأمثلة والأنماط التي تدربت عليها. فهي تقدم نتائج ممتازة للمشكلات الشائعة والمتكررة التي تتوفر لها حلول كثيرة، لكنها تصبح أقل كفاءة عند التعامل مع المشكلات الجديدة أو غير التقليدية التي تتطلب ابتكاراً أو فهماً معمارياً عميقاً، إذ لا يكفي توليد شيفرة صحيحة محلياً، بل يجب أن تنسجم مع التصميم العام للنظام وتحقق جميع متطلباته الوظيفية وغير الوظيفية.



الخلاصة: تعد أدوات الذكاء الاصطناعي وسائل فعالة لزيادة الإنتاجية ومساعدة المطور في كتابة الشيفرة، إلا أنها لا يمكن أن تكون بديلاً عن فهم معمارية النظام. فالمبرمج هو المسؤول عن التأكد من أن جميع الأجزاء المولدة تعمل معاً ضمن بنية موحدة تحقق الأداء، والاعتمادية، وقابلية الصيانة، والتوسع، وهي أمور لا يمكن ضمانها من خلال توليد الشيفرة وحده.

■ ثالثاً: دور المهندس والبرمجيات الوسيطة (Middleware)

بعد استعراض البنيان المؤسساتي (Enterprise Architecture)، يتضح أن دور المهندس البرمجي لا يقتصر على كتابة الشيفرة أو إنشاء الخدمات (Services)، وإنما يتمثل في تطوير خدمات تحقق الوظيفة المطلوبة مع ضمان اندماجها بصورة صحيحة داخل البنية المعمارية للنظام. في الأنظمة المؤسساتية لا تتواصل الخدمات مع بعضها بصورة مباشرة، وإنما يتم ذلك من خلال طبقة Facade التي سبق شرحها. وتمثل هذه الطبقة نقطة الدخول الموحدة (Entry Point) إلى النظام، فهي تستقبل جميع الطلبات القادمة من الخارج، ثم توجهها إلى الخدمة (Service) المناسبة، وتطبق السياسات المشتركة مثل إدارة المناقلات (Transactions)، والأمان (Security)، والتسجيل (Logging)، وغيرها من الجوانب غير الوظيفية. وبناءً على ذلك، فإن مسؤولية المبرمج أو المهندس تنحصر في تطوير الخدمة المطلوبة والتأكد من أنها تتكامل مع طبقة Facade وفق المعمارية المعتمدة، دون أن ينشغل بطريقة وصول الطلب إليها أو بروتوكول الاتصال المستخدم.

دور البرمجيات الوسيطة (Middleware):

- لا تصل الطلبات إلى طبقة Facade مباشرة، وإنما تمر أولاً عبر البرمجيات الوسيطة (Middleware)، التي تشكل حلقة الوصل بين التطبيقات أو الأجهزة المختلفة.
- وقد تدعم طبقة Facade أكثر من نوع من البرمجيات الوسيطة في الوقت نفسه، بحيث يمكن استقبال الطلبات باستخدام تقنيات مختلفة مثل:
 - Sockets
 - RPC (Remote Procedure Call)
 - Java RMI
 - CORBA
 - Enterprise Java Beans (EJB) وغيرها.
- وبغض النظر عن نوع البرمجيات الوسيطة المستخدمة، فإنها جميعاً تؤدي الوظيفة نفسها، وهي نقل الطلبات من العميل (Client) إلى طبقة Facade، التي تتولى بعد ذلك استدعاء الخدمة المناسبة لتنفيذ الطلب.
- وهذا يعني أن الخدمة (Service) نفسها لا تعتمد على نوع البرمجيات الوسيطة المستخدمة، لأنها تحتوي فقط على منطق الأعمال (Business Logic)، بينما تتولى طبقة Facade والـ Middleware مسؤولية استقبال الطلبات ونقلها وتوجيهها.

Keep going.....

اختيار البرمجيات الوسيطة:

- لا يعتمد اختيار نوع البرمجيات الوسيطة على سهولة استخدامها أو على كونها تنتج برنامجاً يعمل فقط، وإنما يجب أن يتم وفق متطلبات النظام غير الوظيفية (Non-functional Requirements).

تذكرة : سبق في المحاضرة الحادية عشرة استعراض الخصائص الأساسية التي يمكن أن توفرها البرمجيات الوسيطة (Middleware)، حيث تم شرح كل خاصية بالتفصيل وبيان أهميتها في الأنظمة الموزعة. وفي هذه المحاضرة أُعيد التأكيد على هذه الخصائص، ولكن من منظور مختلف؛ إذ أصبح التركيز على مقارنة أنواع البرمجيات الوسيطة المختلفة وفق هذه المعايير، وليس على شرح الخصائص نفسها.

و تمت الإشارة الى أنه ليس من الضروري أن يدعم أي Middleware جميع الخصائص، وإنما تختلف الإمكانيات باختلاف الغرض الذي صُمم من أجله.

أمثلة :

1. Function Call قد يؤدي إلى إضعاف توزيع الأحمال (Load Balancing) لأنه يمثل استدعاءً محلياً مباشراً ولا يوفر آليات توزيع الطلبات.
 2. Java RMI يتميز بسهولة الاستخدام، إلا أنه لا يدعم المناقلات (Transactions) بصورة قياسية.
 3. Sockets قد تؤثر سلباً في قابلية التوسع (Scalability) إذا ازداد عدد المستخدمين، إذ يتطلب كل اتصال إدارة مباشرة من التطبيق.
 4. Enterprise Java Beans (EJB) صُممت خصيصاً لدعم الخصائص المؤسسية المهمة مثل المناقلات (Transactions)، وقابلية التوسع (Scalability)، وإدارة الموارد (Resource Management).
- ولهذا السبب، فإن دور المهندس لا يتمثل في كتابة شيفرة تعمل فقط، بل في اختيار البنية والتقنيات التي تحقق متطلبات النظام الحالية والمستقبلية، وتضمن إمكانية توسع النظام وصيانتته واستقراره.

ركزت هذه الفقرة على أن كتابة الخدمات (Services) ليست سوى جزء من عمل المهندس، أما الجزء الأهم فهو دمج هذه الخدمات داخل المعمارية الصحيحة، واختيار البرمجيات الوسيطة المناسبة لتحقيق الخصائص غير الوظيفية للنظام، وهو ما يُعد جوهر تصميم الأنظمة المؤسسية.

ملاحظة (سؤال امتحاني): قد يُطلب في الامتحان تحديد أي نوع من البرمجيات الوسيطة يحقق خاصية معينة، أو أي التقنيات تحقق مجموعة من الخصائص معاً، لذلك ينبغي مراجعة الجدول الذي يقارن بين أنواع الـ Middleware والخصائص التسع المذكورة في المحاضرة السابقة.



جدول مقارنة أنواع البرمجيات الوسيطة وفق الخصائص الاساسية :

■ EJB/J2EE و 3-Tiers Client/Server و Queue

خصائص البرمجيات الوسيطة	Client/Server	3-Tiers Client/Server	Queue	EJB / J2EE
الاستقلال عن منصة العمل	نعم	نعم	Unix، يتوفر على IBM/OS2، Windows NT	نعم
دعم Point-to-Point و Broadcasting	Point to Point only	Point to Point only	نعم	نعم، باستخدام Session Bean و MDB
دعم Transactions	لا	لا	نعم	نعم باستخدام RMI over IIOP
(Reliability) الوثوقية	يعتمد على المبرمج	يعتمد على المبرمج	يضمن وصول الرسائل وذلك بالمحافظة على نسخة منها على القرص الصلب لا تهدم ما لم تعالج الرسالة.	نعم
(Scalability) التصديعية التلاؤم مع تزايد حركة Traffic	لا	نعم	نعم	نعم
دعم التخاطب المتزامن وغير المتزامن	متزامن فقط	متزامن فقط	نعم	نعم
دعم بروتوكول التخاطب	لا	لا	لا	نعم
دعم الاستدعاء بالاسم (Location Transparency)	لا	لا	لا	نعم
تنوع بنى الاتصال	Socket	Socket	نعم	نعم

■ RPC و CORBA و RMI

خصائص البرمجيات الوسيطة	RPC	CORBA	RMI
الاستقلال عن منصة العمل	نعم	نعم	نعم
دعم Point-to-Point / Broadcasting	لا	Point to Point	Point to Point
دعم Transactions	لا	نعم	لا
(Reliability) الوثوقية	نعم	نعم	نعم
(Scalability) التصديعية التلاؤم مع تزايد حركة Traffic	لا	لا	لا
دعم التخاطب المتزامن وغير المتزامن	متزامن فقط	متزامن فقط	متزامن فقط
دعم بروتوكول التخاطب	نعم	نعم	نعم
دعم الاستدعاء بالاسم (Location Transparency)	لا	لا	نعم
تنوع بنى الاتصال	لا	TCP/IP	TCP/IP

■ رابعاً: البرمجة بالمكونات

■ المفاهيم (Classes) / (Frames) / (Components)

سبق التمييز بين الكلاس (Class) والفريم (Frame):

فالكلاس (Class) يمثل الوحدة الأساسية في البرمجة الكائنية، ويحتوي على البيانات (Data) والدوال (Methods) التي تصف كائناً معيناً. أما الفريم (Frame) فهو نوع خاص من الكلاسات، يُستخدم كأساس (Base Class) تُشتق منه عناصر أخرى. ومن الأمثلة الشائعة على ذلك نوافذ واجهات المستخدم (Windows)، حيث يرث كل نوع من النوافذ الخصائص والسلوكيات الأساسية التي يوفرها الـ Frame، ثم يضيف الخصائص التي يحتاجها. بعد ذلك انتقل التطور إلى مفهوم البرمجة بالمكونات (Component-Based Programming)، حيث لا يتعامل المبرمج مع كلاسات عادية فقط، وإنما مع مكونات (Components) تعمل داخل محرك (Engine) أو بيئة تشغيل معينة.

ما هو الـ Component؟

■ الـ Component هو نوع خاص من الكلاسات صُمم ليعمل داخل بيئة تشغيل معينة (Engine أو Framework)، بحيث لا يقتصر دوره على تنفيذ الشيفرة البرمجية، وإنما يصف أيضاً خصائص البيانات والعلاقات بينها.

■ وعند تشغيل التطبيق، يقوم المحرك (Engine) بقراءة هذه المكونات وتحليل المعلومات الموجودة فيها، ثم يتولى تنفيذ كثير من المهام بصورة تلقائية، مثل:

- إنشاء واجهات الاستخدام.
- إنشاء جداول قاعدة البيانات.
- تطبيق القيود (Constraints).
- إدارة العلاقات بين الكائنات.
- تطبيق السياسات المختلفة مثل إدارة المناقلات (Transactions).

وبذلك ينتقل اهتمام المبرمج من كتابة تفاصيل التنفيذ إلى وصف مكونات النظام وخصائصها.

التركيز على بنية المعطيات بدلاً من بنية التحكم:

■ في البرمجة التقليدية كان الجزء الأكبر من العمل يتمثل في كتابة بنية التحكم (Control Structure)، أي كتابة التعليمات التي تحدد ماذا يحدث عند الضغط على زر معين، وكيف يتم التحقق من صحة البيانات، وكيف تنشأ الواجهة وكيف تُحفظ البيانات في قاعدة البيانات.

■ أما في البرمجة بالمكونات، فيصبح التركيز الأساسي على بنية المعطيات (Data Structure)، أي وصف البيانات نفسها والقيود التي تحكمها، بينما يتولى المحرك تنفيذ التفاصيل البرمجية تلقائياً.

ولهذا السبب أصبحت البرمجة بالمكونات أكثر إنتاجية، لأنها تقلل كمية الشيفرة التي يكتبها المبرمج وتسمح له بالتركيز على نموذج البيانات بدلاً من تفاصيل التنفيذ.

دور الـ Annotations (أو Tags):

■ حتى يتمكن المحرك من فهم المكونات، يحتاج إلى معلومات إضافية تصف خصائص كل حقل أو كل مكون.



- ولهذا الغرض تستخدم لغات مثل Java ما يسمى Annotations (Tags)، وهي تعليمات تُكتب فوق الكلاسات أو الحقول لتوضيح خصائصها.

ومن الأمثلة على ذلك:

- أن يكون الحقل فريداً (Unique).
- أن يكون إلزامياً (Mandatory).
- أن يشارك في مناقلة (Transaction).
- أن يمثل علاقة مع كيان آخر.
- تحديد نوع البيانات أو القيود الخاصة بها.

يقوم المحرك بقراءة هذه المعلومات أثناء تشغيل التطبيق، ثم يستخدمها لإنشاء أجزاء كبيرة من النظام بصورة آلية، دون الحاجة إلى كتابتها يدوياً.

مثال (Gender field)

لنفترض وجود كيان يمثل شخصاً، ويحتوي على حقل (Gender).
في البرمجة التقليدية قد يُعرّف هذا الحقل على أنه نص (String)، ثم يضطر المبرمج إلى كتابة جميع التعليمات اللازمة لبناء واجهة الإدخال والتحقق من صحة القيم.

أما في البرمجة بالمكونات، فيكفي تعريف الحقل بحيث يقبل قيمتين فقط:

- رجل.
- امرأة.

وعندما يقرأ المحرك هذا التعريف، يفهم أن عدد القيم محدود، فيختار تلقائياً أداة الإدخال المناسبة، مثل:

- قائمة منسدلة (Combo Box).
- أو أزرار اختيار (Radio Buttons).

دون أن يكتب المبرمج أي شيفرة خاصة ببناء هذه الواجهة.

مثال عدد الأولاد :

إذا احتوى الكيان على حقل يمثل عدد الأولاد، فيمكن للمبرمج تعريف القيود الخاصة به داخل المكون، مثل:

- الحد الأدنى (Minimum).
- الحد الأقصى (Maximum).

وعند قراءة هذه المعلومات، يستطيع المحرك:

- التحقق من صحة القيم المدخلة.
- إنشاء عناصر واجهة الاستخدام المناسبة.
- توليد القيود المقابلة في قاعدة البيانات.
- المحافظة على سلامة البيانات (Integrity).

كل ذلك انطلاقاً من توصيف البيانات فقط، دون الحاجة إلى كتابة تعليمات برمجية منفصلة لكل حالة.



■ الفكرة الأساسية:

الهدف من البرمجة بالمكونات ليس إنشاء كلاسات جديدة فحسب، وإنما وصف البيانات والعلاقات والقيود داخل المكونات، وترك مهمة التنفيذ للمحرك (Engine).

فبدلاً من أن يكتب المبرمج شيفرة لبناء النوافذ، والتحقق من صحة البيانات، وإنشاء الجداول والعلاقات، يكفيه وصف البيانات باستخدام المكونات والـ Annotations، بينما يتولى المحرك توليد هذه الأجزاء وإدارتها تلقائياً.

الخلاصة: في البرمجة التقليدية يركز المبرمج على كيفية تنفيذ البرنامج (Control Structure)، أما في البرمجة بالمكونات فيركز على وصف البيانات (Data Structure)، ويترك للمحرك مهمة تحويل هذا الوصف إلى واجهات استخدام وقواعد بيانات، وآليات تحقق، وبقية تفاصيل التنفيذ.

■ خامساً: تقنيات EJB (Enterprise Java Beans)

تعد EJB - Enterprise Java Beans :

إحدى التقنيات المستخدمة في بناء التطبيقات المؤسسية (Enterprise Applications)، وتندرج ضمن نماذج Service-Oriented Architecture - SOA، حيث تهدف إلى تسهيل تطوير الخدمات وإدارة الموارد وتحقيق قابلية التوسع (Scalability) في التطبيقات الكبيرة.

تعتمد فكرة EJB على فصل منطق الأعمال (Business Logic) عن التفاصيل التقنية المتعلقة بالبنية التحتية، بحيث يركز المبرمج على كتابة الخدمات (Services)، بينما يتولى إطار العمل إدارة الموارد وقواعد البيانات والمناقلات وغيرها من الجوانب التشغيلية.

فصل منطق الأعمال عن التفاصيل التقنية

من أهم الأفكار التي تقوم عليها تقنية EJB أن المبرمج لا يحدد داخل الشيفرة البرمجية تفاصيل مثل:

- قاعدة البيانات التي ستُخزن فيها البيانات.
- نوع المناقلة (Transaction).
- مخطط قاعدة البيانات (Schema).
- طوابير الرسائل (Queues).

بل يكفي بكتابة منطق الأعمال (Business Logic)، بينما تُحدد هذه التفاصيل لاحقاً أثناء عملية النشر (Deployment).

ويتم ذلك من خلال ملف خاص يسمى Deployment Descriptor، يحتوي على إعدادات تشغيل التطبيق، ويحدد كيفية ربط المكونات مع قواعد البيانات، وآلية إدارة المناقلات، وغيرها من الإعدادات التشغيلية.

ولهذا السبب يقال إن EJB تفصل "المخ عن العضلات"؛ فالمبرمج يكتب المنطق البرمجي فقط، أما تفاصيل التنفيذ الفعلية فتُحدد عند نشر التطبيق.

فعلى سبيل المثال، قد يكتب المبرمج مكوناً يمثل كياناً معيناً، ثم يقرر مسؤول النشر (Deployment) أثناء

تشغيل التطبيق:

1. قاعدة البيانات التي سيستخدمها.
2. الـ Schema المناسبة.

3. طريقة الربط مع قاعدة بيانات موجودة مسبقاً دون الحاجة إلى إنشاء جداول جديدة.
4. وبذلك يمكن تشغيل التطبيق نفسه في بيئات مختلفة دون تعديل الشيفرة البرمجية.

إعادة استخدام الموارد (Pooling)

تعتمد EJB أيضاً على مبدأ Pooling، وهو إنشاء عدد محدود من الكائنات (Objects) وإعادة استخدامها لخدمة الطلبات المختلفة، بدلاً من إنشاء كائن جديد لكل مستخدم أو لكل طلب.

وتؤدي هذه الآلية إلى:

- تقليل استهلاك الذاكرة.
- تخفيض الزمن اللازم لإنشاء الكائنات.
- تحسين أداء النظام.
- زيادة قدرة التطبيق على خدمة أعداد كبيرة من المستخدمين.

ولهذا تُعد الـ Pooling إحدى الوسائل الأساسية التي تحقق قابلية التوسع (Scalability) في التطبيقات المؤسسية.

أنواع Enterprise Java Beans

تنقسم EJB إلى ثلاثة أنواع رئيسية، ويُعد هذا التصنيف من النقاط المهمة التي يجب معرفتها.

1. Session Beans

تُستخدم لتغليف الخدمات (Services) ومنطق الأعمال (Business Logic)، وهي المسؤولة عن تنفيذ العمليات التي يطلبها المستخدم.

وتنقسم إلى نوعين:

1. Stateless Session Bean: لا تحتفظ بأي معلومات عن المستخدم بعد انتهاء تنفيذ الطلب، لذلك يمكن إعادة استخدامها مع مستخدمين مختلفين.
2. Stateful Session Bean: تحتفظ بحالة المستخدم (State) بين الطلبات المختلفة، وتستخدم عندما تحتاج الخدمة إلى تذكر معلومات خاصة بالمستخدم أثناء جلسة العمل.

2. Entity Beans

تُستخدم لتغليف الكيانات (Entities) التي تُخزن في قواعد البيانات، وتمثل الربط بين الكائنات البرمجية والبيانات المخزنة. وتتولى إدارة عمليات القراءة والكتابة والتخزين دون أن يضطر المبرمج للتعامل مباشرة مع تفاصيل قاعدة البيانات.

3. Message-Driven Beans

تُستخدم للتعامل مع طوابير الرسائل (Message Queues)، حيث تستقبل الرسائل المرسلة إلى الطوابير وتعالجها بصورة غير متزامنة (Asynchronous).

وبذلك يتم تغليف جميع تفاصيل التعامل مع الـ Queue داخل هذا النوع من الـ Beans.

1. ملاحظة (سؤال امتحاني): يجب معرفة الأنواع الثلاثة لـ Enterprise Java Beans ووظيفة كل نوع:

1. Session Bean → تغليف الخدمات (Services)، وتنقسم إلى Stateless و Stateful.
2. Entity Bean → تغليف الكيانات التي تُخزن في قواعد البيانات.
3. Message-Driven Bean → تغليف طوابير الرسائل (Queues).

العلاقة مع المناقلات التصريحية (Declarative Transactions)

ترتبط تقنية EJB مباشرةً بالمناقلات التصريحية (Declarative Transactions)، إذ لا يقوم المبرمج عادةً بتحديد نوع المناقلة أو آلية إدارتها داخل الشيفرة البرمجية، وإنما تحدد هذه الإعدادات أثناء عملية النشر من خلال إعدادات التطبيق.

وبذلك يمكن تغيير طريقة إدارة المناقلات أو قاعدة البيانات المستخدمة دون الحاجة إلى تعديل الكود، وهو ما يحقق فصلاً واضحاً بين منطق الأعمال والتفاصيل التشغيلية، ويزيد من مرونة التطبيقات المؤسساتية وسهولة صيانتها.

سادساً: تطور تقنيات التواصل الموزع

شهدت تقنيات التواصل بين التطبيقات تطوراً مستمراً بهدف تبسيط عملية بناء الأنظمة الموزعة، مع المحافظة على الأداء والموثوقية وتقليل التعقيد البرمجي.

بدأت هذه التقنيات بحلول مثل Java RMI و CORBA، ثم تطورت إلى Web Services، وصولاً إلى الأسلوب الأكثر انتشاراً حالياً، وهو REST APIs المعتمد على بروتوكول HTTP وتنسيق JSON.

1. Java RMI :

- وفرت تقنية (Remote Method Invocation) Java RMI أسلوباً بسيطاً لاستدعاء الكائنات الموجودة على أجهزة أخرى، وجعلت البرمجة الموزعة أسهل من التقنيات السابقة.
- إلا أنها كانت محدودة ببيئة Java، كما أنها لا توفر دعماً مباشراً للمناقلات الموزعة (Distributed Transactions)، الأمر الذي جعل استخدامها محدوداً في التطبيقات المؤسساتية الكبيرة.

2. CORBA و EJB :

- بعد ذلك ظهرت CORBA، التي وفرت إمكانيات أكبر، مثل دعم التطبيقات المكتوبة بلغات برمجة مختلفة، إضافة إلى دعم المناقلات الموزعة.
- وللتغلب على صعوبة استخدام CORBA، ظهرت Enterprise Java Beans (EJB)، التي جمعت بين سهولة برمجة Java RMI والإمكانات التي توفرها CORBA، مثل دعم المناقلات وإدارة الموارد.

3. Web Services :

- مع انتشار تطبيقات الويب ظهرت Web Services، والتي اعتمدت الفكرة نفسها المستخدمة في RPC، ولكن عبر بروتوكولات الويب بدلاً من الشبكات المحلية (LAN).

- فمن الناحية المفاهيمية لا تختلف Web Services كثيراً عن RPC، إذ تقوم أيضاً على استدعاء خدمة بعيدة (Remote Service)، إلا أن التواصل يتم عبر بروتوكولات الويب، مما يسمح بتبادل الخدمات عبر الإنترنت بدلاً من الاقتصار على الشبكات المحلية.
- وقد جمعت Web Services بين سهولة الاستخدام التي وفرتها Java RMI والإمكانات التي كانت توفرها RPC، إلا أنها عانت من مشكلة رئيسية تمثلت في تعقيد عملية النشر (Deployment)، حيث كان نشر الخدمات وإعدادها يتطلب خطوات كثيرة وإعدادات معقدة، إضافة إلى الحاجة إلى تعريف كل خدمة (Service) بصورة مستقلة.

4. REST APIs :

- نتيجة لتعقيد Web Services، اتجه المطورون إلى أسلوب أبسط يعتمد مباشرةً على بروتوكول HTTP وتبادل البيانات باستخدام JSON، وهو ما أصبح يعرف باسم REST APIs.
- يعتمد REST على إرسال طلبات HTTP وإرجاع البيانات بصيغة JSON، دون الحاجة إلى البنية المعقدة التي كانت تتطلبها Web Services، مما جعل تطوير الخدمات ونشرها أسهل وأكثر مرونة.
- ولهذا السبب أصبح REST الأسلوب الأكثر استخداماً في تطوير التطبيقات الحديثة وواجهات البرمجة (APIs)، خاصة في تطبيقات الويب والهواتف الذكية.

التوجه نحو البرمجة المرنة

رافق هذا التطور أيضاً تغير في فلسفة تصميم البرمجيات، إذ اتجهت التطبيقات الحديثة إلى تقليل الاعتماد على التحقق الصارم من الأنواع (Strong Type Checking)، والاعتماد بصورة أكبر على صيغ بيانات مرنة مثل JSON، ولغات ديناميكية مثل JavaScript. وقد ساهم هذا التوجه في زيادة سرعة التطوير وتبسيط التكامل بين الأنظمة المختلفة، خصوصاً في تطبيقات الويب الحديثة التي تعتمد بصورة أساسية على REST وJSON.

- **الخلاصة:** تطورت تقنيات الاتصال من RMI إلى CORBA/EJB ثم إلى Web Services، وأخيراً إلى REST APIs، وكان الهدف من هذا التطور هو تقليل التعقيد، وتسهيل النشر (Deployment)، والاعتماد على بروتوكولات الويب القياسية مثل HTTP وتنسيق JSON لتبادل البيانات بين الأنظمة.

إلى هنا نكون قد أنهينا رحلتنا معكم في البرمجة التفرعية، ونأمل أن يكون عملنا قد ساعدكم

على فهم المادة بشكل أفضل.

نتمنى لكم كل النجاح والتوفيق في امتحاناتكم القادمة، ولا تنسونا من صالح دعائكم.

-- فريق البرمجة التفرعية --

